

Software de comunicaciones

Curso 2026/2027

Práctica 1

Distribución de una aplicación usando RPCs

Indice

Objetivos 1

Entrega de resultados 1

Enunciado..... 1

 Fase 1..... 3

 Fase 2..... 4

 Fase 3..... 5

Anexo 1..... 7

Anexo 2..... 8

Anexo 3..... 9

Anexo 4..... 10

Objetivos

- Entender los fundamentos de la distribución de aplicaciones utilizando middleware.
- Desarrollar una aplicación distribuida utilizando invocación remota de procesos de servicio (RPC) usando gRPC.

Entrega de resultados

Se debe entregar un fichero zip en la tarea correspondiente de Moodle con los siguientes contenidos:

1. Carpetas completas de TODOS los proyectos Maven creados en la realización de la práctica. Las carpetas entregadas deben contener todos los ficheros fuente y los creados por las herramientas.
2. Archivo de texto con las respuestas a todas las preguntas numeradas realizadas en este enunciado.

La entrega en Moodle debe realizarse ANTES del 3 de noviembre.

Los resultados entregados para esta práctica podrán ser necesarios para la realización del examen práctico del primer parcial de la asignatura.

Enunciado

Se dispone de una aplicación que implementa una calculadora con memoria. Esta aplicación tiene un funcionamiento monolítico cuyo diagrama de clases se muestra en el Anexo 1.

La aplicación calculadora dispone de las cuatro operaciones aritméticas básicas, funciones programables, así como de la capacidad de mostrar el último resultado obtenido y de utilizar una memoria acumuladora. La calculadora dispone de una interfaz gráfica de usuario (Graphical User Interface, GUI) que se presenta a continuación.



La calculadora dispone de dos áreas diferenciadas: la de presentación de resultados en la parte superior y los botones en la parte inferior. El área de representación dispone de dos líneas: una para visualizar las operaciones realizadas y una segunda línea para la visualización de los números que se componen y los resultados obtenidos. Los botones, por su parte, son de tres clases: los que permiten componer un número dígito a dígito, los operadores aritméticos y los de manipulación de las memorias. Las dos primeras clases permiten una operativa evidente, así que no se presenta su funcionalidad. En cuanto a los botones de manipulación de memorias, esta es su funcionalidad:

- **Último resultado (UR):** Permite visualizar el resultado de la última operación aritmética realizada.
- **Memoria-Añadir (MA):** Permite añadir a la memoria acumuladora el resultado de la última operación aritmética realizada.
- **Memoria-Obtener (MO):** Permite visualizar el contenido de la memoria acumuladora.
- **Memoria-Limpiar (ML):** Permite poner a cero el contenido de la memoria acumuladora.

Existe un botón adicional **Borrar Entrada (BE)**, que permite borrar la entrada actual en caso de cometer un error al componer un número.

Esta práctica se debe realizar siguiendo en secuencia cada una de sus fases.

En la primera fase el alumno debe implementar primero las clases que permitan ofrecer la funcionalidad completa de la aplicación, teniendo especial cuidado en la implementación de la clase **OperacionesCalculadora**. Una vez disponible la aplicación se debe verificar el funcionamiento correcto, de acuerdo con lo esperado.

Con posterioridad, en las fases 2 y 3 se realizará una distribución de la aplicación, de tal manera que la capacidad de realizar las operaciones, así como la gestión de las memorias de la calculadora, pasarán a ser hospedadas en un servicio de un servidor específico.

Fase 1

En esta fase, se deben codificar completamente las clases de la aplicación correspondientes con el paquete **calculadora**: **AplicacionCalculadoraGUI**, **OperacionesCalculadora** y **AdaptadorOperacionesCalculadoraGUI** (adaptador al GUI).

Se debe obtener vía Github el modelo de proyecto Maven **p1-calculadora** (ver Anexo 4) que incluye los siguientes elementos:

1. Clases del paquete **calculadora** que se han codificado.
2. Fichero **pom.xml** con la configuración necesaria básica para poder construir la aplicación usando el paquete **CalculadoraGUI** que se ofrece ya implementado en un repositorio.

Es necesario realizar las pruebas oportunas para verificar que las implementaciones de las clases son coherentes con las funcionalidades requeridas para la calculadora. Para realizar pruebas exhaustivas de la implementación de **OperacionesCalculadora** y **AdaptadorOperacionesCalculadoraGUI**, se proporciona la clase **Implementacion_ICalculadora_Test** que contiene pruebas [JUnit 5](#). No se debe continuar con la siguiente fase hasta que no se haya terminado completamente esta.

NOTAS:

- a. La clase **OperacionesCalculadora** debe disponer de los mismos métodos especificados en la interfaz **CalculadoraGUI.ICalculadora**. Cada método debe llevar el prefijo 'implementación'.
- b. Aunque es posible implementar la aplicación evitando el uso de la clase adaptadora **AdaptadorOperacionesCalculadoraGUI**, usando sólo la clase **OperacionesCalculadora**, en esta fase se mantendrá la interfaz de esta clase para su uso en las siguientes fases.

Fase 2

En esta fase se debe realizar la distribución de la aplicación. Para ello se seguirán los siguientes pasos:

1. Se debe obtener vía Github el modelo de proyecto Maven cliente de nombre **p1-cliente1**, que incluye los siguientes elementos:
 - a. Clases del paquete **cliente**:
 - i. Clase **AdaptadorOperacionesCalculadoraGUI** que implementa el adaptador entre el GUI y el stub cliente de gRPC.
 - ii. Clase **ClienteCalculadoraGUI** que implementa la aplicación.
 - iii. Fichero **calculadora.proto**
 - b. **pom.xml** con la configuración necesaria básica para poder construir la aplicación usando las bibliotecas de gRPC, protobuf y el paquete **CalculadoraGUI** que se ofrece ya implementado en un repositorio.
2. Se describirá un servicio utilizando el lenguaje de definición de interfaz (Interface Definition Language, IDL) de gRPC/protobuf.
 - a. El fichero que describe el servicio tiene el nombre **calculadora.proto** y está colocado en el directorio **src/main/proto** del proyecto.
 - b. El nombre del servicio será **ServicioCalculadora** (exactamente este nombre, letra a letra).
 - c. El nombre del espacio de nombres (nombre del paquete java que se va a generar) será **stubCalculadora** (exactamente este nombre, letra a letra).
 - d. Cada método de la interfaz **CalculadoraGUI.ICalculadora** se describirá como un procedimiento del servicio.
 - e. Se deben definir todos los tipos de mensajes necesarios para los procedimientos.
3. Se realizará la compilación del servicio usando el plugin de Maven **protobuf-maven-plugin**, que ya se incluye configurado en **pom.xml**.
4. Se debe obtener vía Github el modelo de proyecto Maven cliente de nombre **p1-servidor1**, que incluye los siguientes elementos:
 - a. Clases del paquete **servidor**:
 - i. Clase **AdaptadorOperacionesCalculadora** que implementa los procedimientos que manejan las operaciones del servicio.
 - ii. Clase **ServidorCalculadora** que implementa el servidor gRPC multihilo.
 - b. **pom.xml** con la configuración necesaria básica para poder construir la aplicación usando las bibliotecas de gRPC y protobuf.
5. Se deben añadir los siguientes archivos:
 - a. Clases del paquete **calculadora** ya implementado en la fase 1:
 - i. Clase **OperacionesCalculadora** que implementa las operaciones.
 - b. Se debe copiar **EL MISMO** fichero **calculadora.proto** que describe el servicio desarrollado para el proyecto cliente en el directorio **src/main/proto** del proyecto del servidor.
6. Se debe probar el cliente y servidor para verificar la funcionalidad en todos los casos, incluyendo aquellos en los que las operaciones remotas arrojan excepciones.
7. Se debe intentar lanzar un segundo cliente mientras se está atendiendo a las peticiones del primero.

PREGUNTAS 1 y 2: ¿Qué ocurre cuando el segundo cliente intenta solicitar operaciones remotas? ¿Por qué ocurre eso?

Se deben incluir las respuestas a estas preguntas como comentarios iniciales del archivo `ServidorCalculadora.java`.

NOTAS:

- a. Los diagramas de clases correspondientes con las aplicaciones cliente y servidor se pueden consultar en los Anexos 2 y 3.1.

Fase 3

En esta fase se debe implementar un servicio que es capaz de asociar una instancia de **OperacionesCalculadora** a cada cliente. Se debe probar su funcionamiento con dos o más clientes concurrentes.

Para poder asociar la instancia de **OperacionesCalculadora** que corresponde a cada cliente se va a utilizar el [patrón Estado de Sesión en Servidor \(*Server Session State*\) que usa un Identificador de Sesión \(*session ID*\) único para cada cliente.](#)

Para realizar esta fase, se van a construir un nuevo cliente y servidor basados en los ya disponibles en la fase anterior. Para ello se seguirán los siguientes pasos:

1. Realizar una copia del proyecto **p1-cliente1** y cambiar su nombre para que se llame **p1-cliente2**.
2. Modificar el **artifactId** en `pom.xml` del nuevo proyecto para que coincida con el nombre de dicho proyecto.
3. Modificar `calculadora.proto` para que:
 - a. cada cliente pueda obtener del servicio el Identificador de Sesión que se corresponda con el estado de la calculadora que usa en exclusiva y
 - b. que cada petición de procedimiento remoto incluya dicho Identificador de Sesión, de tal manera que el procedimiento remoto utilice el estado de la calculadora asociado al cliente.
4. Modificar la implementación de **AdaptadorOperacionesCalculadoraGUI** en coherencia con la interfaz definida.
5. Se debe obtener vía Github el modelo de proyecto Maven cliente de nombre **p1-servidor2**, que incluye los siguientes elementos:
 - a. Clases del paquete **persistencia**:
 - i. Clase **CalculadoraDAO_Mapa** que implementa el almacén concreto Data Access Object.
 - ii. Interfaces **ICalculadoraDAO** y **DAO**.
 - b. `pom.xml` con la configuración necesaria básica para poder construir la aplicación usando las bibliotecas de gRPC y protobuf.
6. Copiar en dicho proyecto las implementaciones de las clases de los paquetes **calculadora** y **servidor** y `calculadora.proto` de la fase 2.
7. Modificar la implementación de **AdaptadorOperacionesCalculadora** en coherencia con la interfaz definida en `calculadora.proto`.

8. Se debe probar el cliente y servidor de esta fase para verificar la funcionalidad en todos los casos, incluyendo aquellos en los que las operaciones remotas arrojan excepciones.
9. Se debe intentar lanzar un segundo cliente mientras se está atendiendo a las peticiones del primero.

PREGUNTAS 3 y 4: ¿Qué observa si en un cliente realiza una operación y en el otro solicita visualizar el último resultado? ¿Por qué ocurre eso?

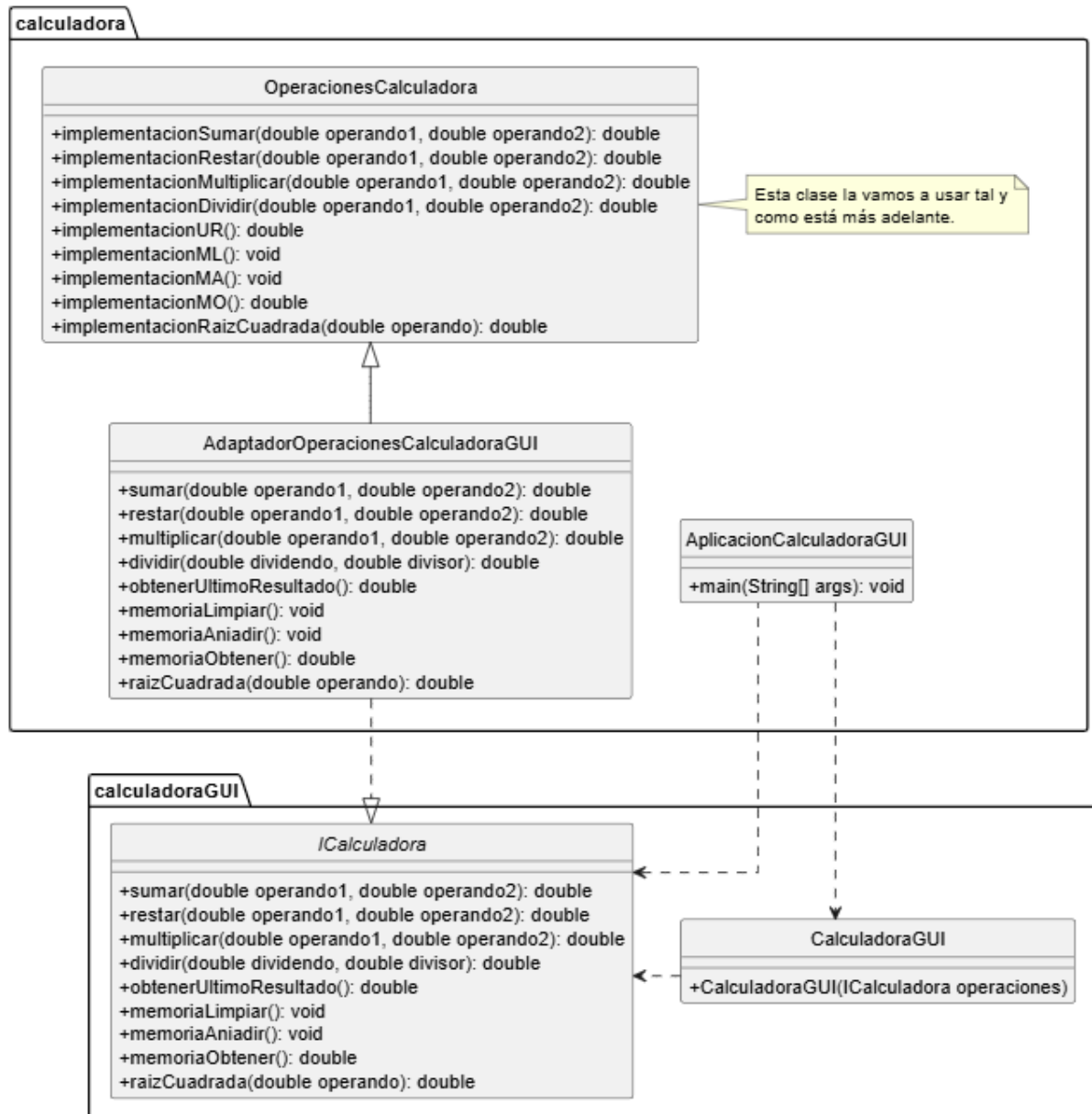
Se deben incluir las respuestas a estas preguntas como comentarios iniciales del archivo `ServidorCalculadora.java`.

NOTAS:

- a. Los diagramas de clases correspondientes con las aplicaciones cliente y servidor se pueden consultar en los Anexos 2 y 3.2.

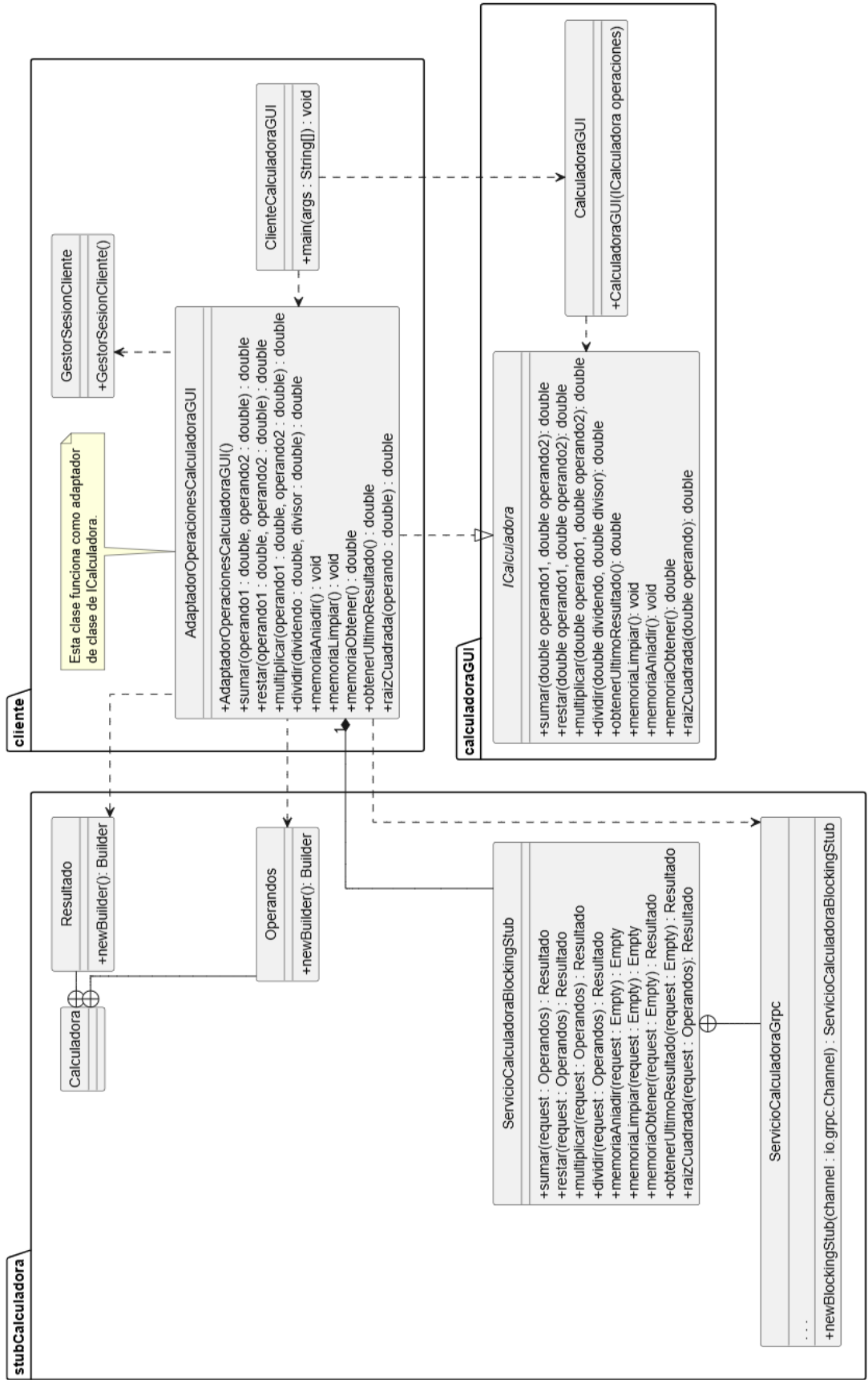
Anexo 1

Diagrama de clases de la aplicación calculadora con memoria.



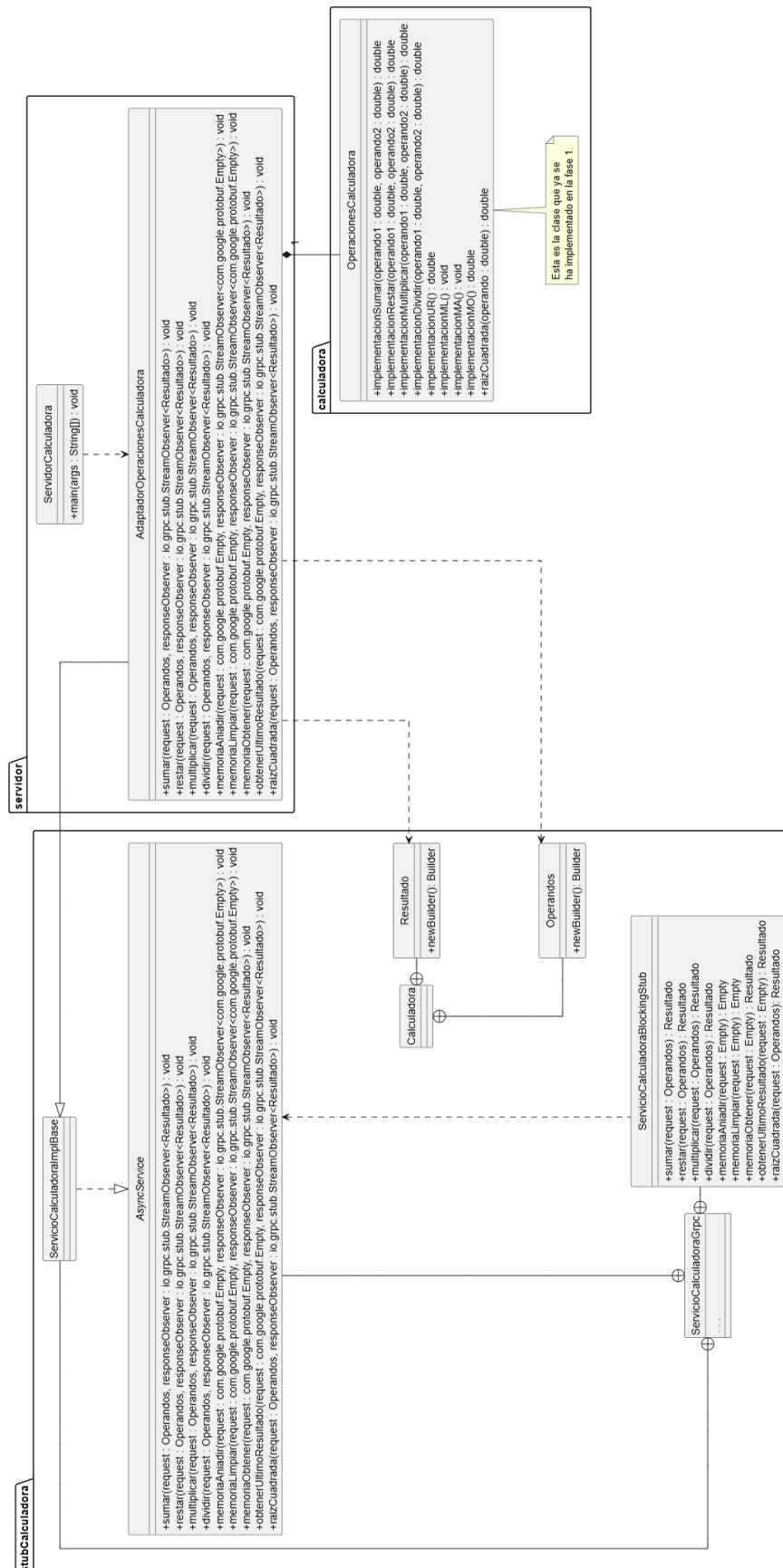
Anexo 2

Diagrama de clases de la aplicación cliente calculadora con memoria.



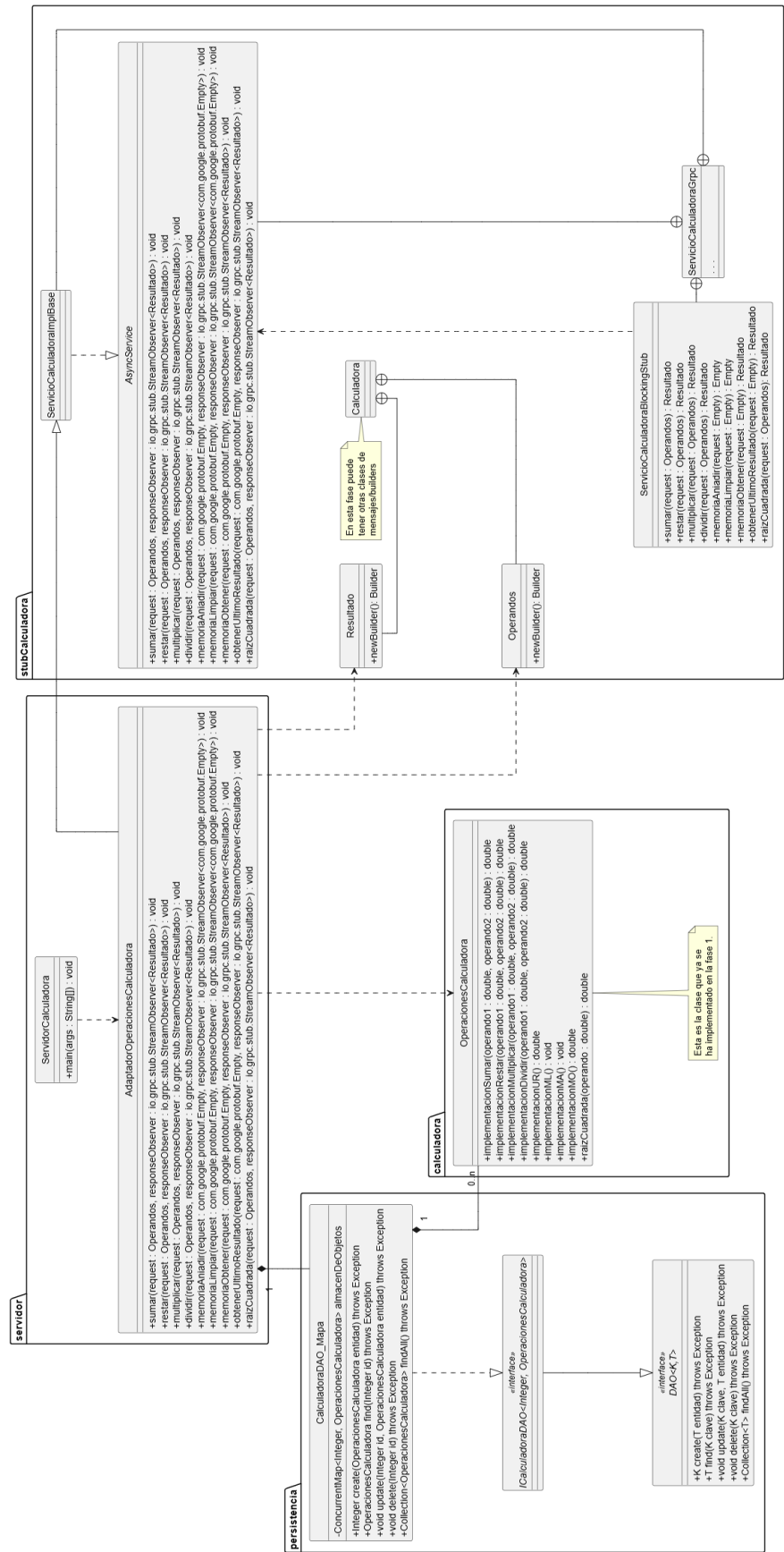
Anexo 3.1

Diagrama de clases de la aplicación servidor calculadora con memoria de la Fase 2.



Anexo 3.2

Diagrama de clases de la aplicación servidor calculadora con memoria de la Fase 3.



Anexo 4

Acceso al repositorio de Github de la asignatura para la obtención de los modelos de proyecto de la práctica 1: **p1-calculadora**, **p1-cliente1**, **p1-servidor1**. y **p1-servidor2**

URL: https://github.com/sof-com/sof-com-repo.git
--